

Übungsblatt 10
Vorlesung “Cybersicherheit”
Sommersemester 2025
Praktische Einführung in C

Einführung: Die Kompilierungsphasen eines C-Programms

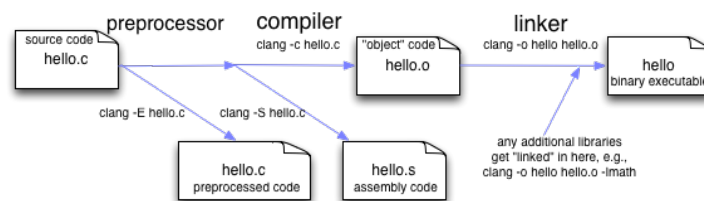


Abbildung 1: Kompilierungsphasen am Beispiel eines HalloWelt Programs in C

Ein C-Programm durchläuft mehrere Verarbeitungsschritte, bevor es als ausführbare Datei vorliegt. Dieser Prozess, bekannt als **Kompilierungspipeline**, besteht typischerweise aus vier Hauptphasen:

1. **Präprozessor-Phase** - Der Präprozessor verarbeitet Direktiven wie `#include` und `#define` und erzeugt erweiterten Quellcode.
2. **Kompilierungsphase** - Der Compiler übersetzt den vorverarbeiteten Code in Assemblerbefehle (architekturspezifisch).
3. **Assemblerphase** - Der Assembler wandelt den Assembler-Code in maschinenlesbaren Objektcode (`.o` oder `.obj`).
4. **Linker-Phase** - Der Linker kombiniert Objektdateien und Bibliotheken zur finalen ausführbaren Datei.

Diese Schritte gewährleisten, dass aus lesbarem C-Quellcode ein funktionsfähiges Programm entsteht. Tools wie `gcc` automatisieren diesen Ablauf, doch das Verständnis der einzelnen Phasen ist essenziell für Debugging und Optimierung.

Beispiel mit GCC:

```
gcc -o program program.c # Führt alle Schritte aus
```

Weitere Details zum Thema: <https://jsommers.github.io/cbook/programstructure.html>.

Hello World Programm in C

```
1 #include <stdio.h>
2
3 int main() {
4     // Ausgabe des Textes "Hello, World!" auf dem Bildschirm
5     printf("Hello, World!\n");
6     return 0; // Rueckgabewert
7 }
```

Erklärung der main-Funktion

- `int main()`
 - Ist der Einstiegspunkt jedes C-Programms
 - Wird vom Betriebssystem beim Programmstart aufgerufen
 - Der Rückgabebetyp `int` zeigt an, dass ein Integer-Wert zurückgegeben wird
- `return 0;`
 - Signalisiert erfolgreiche Beendigung (0 = Erfolg)
 - Andere Werte können Fehler anzeigen
- `#include <stdio.h>`
 - Bindet die Standard-Input/Output-Bibliothek ein
 - Erforderlich für Ein-/Ausgabefunktionen wie `printf()`
- `printf("Hello, World!\n");`
 - Gibt Text auf der Konsole aus
 - `\n` erzeugt einen Zeilenumbruch

Kompilieren und Ausführen

```
1 gcc HelloWorld.c -o HelloWorld # Kompilieren
2 ./HelloWorld # Ausfuehren
```

Ausgabe:

Hello, World!

Die `main`-Funktion ist obligatorisch und steuert den Programmfluss. Jedes C-Programm muss genau eine `main`-Funktion enthalten.

Grundlagen der Variablendeklaration in C

In C müssen Variablen vor ihrer Verwendung deklariert werden. Eine Variablendeklaration besteht aus:

- Datentyp
- Variablenname
- Optional: Initialisierung mit einem Wert

Codebeispiele

```
1 #include <stdio.h>
2
3 int main() {
4     // Einfache Deklarationen
5     int alter;           // Ganze Zahl
6     float preis;         // Fließkommazahl
7     char buchstabe;      // Einzelnes Zeichen
8
9     // Deklaration mit Initialisierung
10    int stunden = 24;     // Direkte Wertzuweisung
11    float pi = 3.14159;
12    char gruppe = 'A';
13
14    // Mehrere Variablen gleichen Typs deklarieren
15    int x, y, z;          // Drei Integer-Variablen
16    float temp1, temp2;   // Zwei Float-Variablen
17
18    // Wertezuweisung nach Deklaration
19    alter = 30;
20    preis = 19.99;
21    buchstabe = 'B';
22
23    // Ausgabe der Variablen
24    printf("Alter: %d\n", alter);
25    printf("Preis: %.2f\n", preis);
26    printf("Buchstabe: %c\n", buchstabe);
27
28    return 0;
29 }
```

Wichtige Regeln

- Variablennamen müssen mit einem Buchstaben oder Unterstrich beginnen
- C ist case-sensitiv (alter und Alter sind unterschiedlich)
- Reservierte Schlüsselwörter dürfen nicht als Namen verwendet werden
- Variablen haben lokalen Gültigkeitsbereich (innerhalb der Blockklammern)

Gängige Datentypen

Datentyp	Größe	Beschreibung
int	4 Bytes	Ganze Zahlen
float	4 Bytes	Fließkommazahl
double	8 Bytes	Große Fließkommazahl
char	1 Byte	Einzelnes Zeichen

Verzweigungen und Fallunterscheidungen in C

Verzweigungen ermöglichen es, den Programmfluss basierend auf Bedingungen zu steuern. In C gibt es mehrere Möglichkeiten für Fallunterscheidungen:

if-Anweisung

Die einfachste Form der Verzweigung:

```
1 if (bedingung) {
2     // Codeblock, der ausgeführt wird wenn die Bedingung wahr ist
3 }
```

Beispiel:

```
1 int alter = 18;
2 if (alter >= 18) {
3     printf("Sie sind volljaehrig.\n");
4 }
```

if-else-Anweisung

Zweigeteilte Verzweigung:

```
1 if (bedingung) {
2     // Codeblock wenn wahr
3 } else {
4     // Codeblock wenn falsch
5 }
```

Beispiel:

```
1 int zahl = 10;
2 if (zahl % 2 == 0) {
3     printf("Die Zahl ist gerade.\n");
4 } else {
5     printf("Die Zahl ist ungerade.\n");
6 }
```

else-if-Kaskade

Für mehrfache Verzweigungen:

```
1 if (bedingung1) {
2     // Codeblock 1
3 } else if (bedingung2) {
4     // Codeblock 2
5 } else {
6     // Codeblock wenn keine Bedingung zutrifft
7 }
```

Beispiel:

```
1 int note = 2;
2 if (note == 1) {
3     printf("Sehr gut!\n");
4 } else if (note == 2) {
5     printf("Gut!\n");
6 } else if (note == 3) {
7     printf("Befriedigend\n");
8 } else {
9     printf("Andere Note\n");
10 }
```

Switch-Case-Anweisung

```
1 switch (ausdruck) {  
2     case konstante1:  
3         // Codeblock 1  
4         break;  
5     case konstante2:  
6         // Codeblock 2  
7         break;  
8     default:  
9         // Standard-Code  
10 }
```

Beispiel:

```
1 char op = '+';  
2 switch (op) {  
3     case '+': printf("Addition\n"); break;  
4     case '-': printf("Subtraktion\n"); break;  
5     default: printf("Unbekannte Operation\n");  
6 }
```

Wichtige Hinweise

- Bedingungen müssen in Klammern stehen
- Codeblöcke werden mit geschweiften Klammern {} umschlossen
- break verhindert Fall-Through im Switch
- default-Zweig ist optional

Vergleichsoperatoren

- ==, !=, <, >, <=, >=

Logische Operatoren

- && (UND)
- || (ODER)
- ! (NICHT)

Beispiel:

```
1 if (alter > 12 && alter < 20) {  
2     printf("Jugendlicher\n");  
3 }
```

Schleifen in C, eine Kurzfassung

While-Schleife

Die `while`-Schleife wiederholt einen Codeblock, solange eine Bedingung wahr ist.

```
1 while (bedingung) {  
2     // Codeblock  
3 }
```

Beispiel:

```
1 int zaehler = 0;  
2 while (zaehler < 5) {  
3     printf("Zaehler: %d\n", zaehler);  
4     zaehler++;  
5 }
```

Do-While-Schleife

Die `do-while`-Schleife führt den Codeblock mindestens einmal aus und überprüft dann die Bedingung.

```
1 do {  
2     // Codeblock  
3 } while (bedingung);
```

Beispiel:

```
1 int zahl;  
2 do {  
3     printf("Positive Zahl eingeben: ");  
4     scanf("%d", &zahl);  
5 } while (zahl <= 0);
```

For-Schleife

Die `for`-Schleife kombiniert Initialisierung, Bedingung und Inkrement in einer Zeile.

```
1 for (initialisierung; bedingung; inkrement) {  
2     // Codeblock  
3 }
```

Beispiel:

```
1 for (int i = 0; i < 10; i++) {  
2     printf("%d ", i);  
3 }
```

Schleifensteuerung

- `break` - Beendet die Schleife sofort
- `continue` - Überspringt den Rest des aktuellen Durchlaufs

Beispiel:

```
1 for (int i = 0; i < 10; i++) {  
2     if (i == 5) break; // Schleife bei 5 beenden  
3     if (i % 2 == 0) continue; // Ungerade Zahlen ueberspringen  
4     printf("%d ", i);  
5 }
```

Endlose Schleifen

Eine Schleife ohne Endbedingung läuft unendlich:

```
1 while (1) {  
2     // Endlos wiederholter Code  
3     if (abbruchbedingung) break;  
4 }
```

Vergleich der Schleifen

Schleifentyp	Einsatzzweck
while	Wenn Anzahl Durchläufe unbekannt
do-while	Mindestens ein Durchlauf nötig
for	Bekannte Anzahl Durchläufe

Grundlagen von Funktionen

Funktionen sind Codeblöcke, die eine spezifische Aufgabe erfüllen und wiederverwendbar sind.

Funktionsaufbau

```
1 Rueckgabebetyp Funktionsname(Parameterliste) {  
2     // Funktionskoerper  
3     return wert; // optional bei void  
4 }
```

Funktionsdeklaration vs. Definition

- **Deklaration** (Prototyp): Gibt Signatur bekannt
- **Definition**: Implementiert die Funktion

Beispiel:

```
1 // Deklaration  
2 int summe(int a, int b);  
3  
4 // Definition  
5 int summe(int a, int b) {  
6     return a + b;  
7 }
```

Parameterübergabe

Die Programmiersprache C kennt zwei Arten der Parameterübergabe:

Wertübergabe (Call by Value)

```
1 void quadrat(int x) {  
2     x = x * x;  
3 }
```

Referenzübergabe (Call by Reference)

```
1 void quadrat(int *x) {  
2     *x = (*x) * (*x);  
3 }
```

Rückgabewerte

- void für keine Rückgabe
- Beliebige Datentypen möglich
- Nur ein Rückgabewert direkt möglich

Beispiel:

```
1 float durchschnitt(float a, float b) {  
2     return (a + b) / 2;  
3 }
```


Rekursion

Funktionen können sich selbst aufrufen:

```
1 int fakultaet(int n) {  
2     if (n <= 1) return 1;  
3     return n * fakultaet(n - 1);  
4 }
```

Scope von Variablen

- **Lokale Variablen:** Nur in Funktion sichtbar
- **Globale Variablen:** Außerhalb aller Funktionen

Header-Dateien

Funktionsprototypen werden typischerweise in Header-Dateien ausgelagert:

```
1 // mathe.h  
2 #ifndef MATHE_H  
3 #define MATHE_H  
4  
5 int summe(int a, int b);  
6 float durchschnitt(float a, float b);  
7  
8 #endif
```

Beispielprogramm

```
1 #include <stdio.h>  
2  
3 // Prototyp  
4 void begruessung(char name[]);  
5  
6 int main() {  
7     begruessung("Anna");  
8     return 0;  
9 }  
10  
11 // Funktionsdefinition  
12 void begruessung(char name[]) {  
13     printf("Hallo, %s!\n", name);  
14 }
```

Best Practices

- Eine Funktion sollte nur eine Aufgabe erfüllen
- Gute Funktionsnamen wählen (was tut die Funktion?)
- Dokumentation mit Kommentaren
- Möglichst wenige globale Variablen verwenden
- Parameter und Rückgabewerte sinnvoll wählen

Arrays und Pointer in C

Arrays in C

Arrays sind zusammenhängende Speicherbereiche für mehrere Werte desselben Typs.

Deklaration und Initialisierung

```
1 // Deklaration
2 int zahlen[5];           // Uninitialisiertes Array
3 double werte[3] = {1.1, 2.2, 3.3}; // Initialisiert
4
5 // Automatische Groessenbestimmung
6 char buchstaben[] = {'a', 'b', 'c'}; // Groesse 3
```

Zugriff auf Elemente

```
1 zahlen[0] = 10;          // Erstes Element (Index 0)
2 int x = zahlen[2];       // Drittes Element
```

Pointer in C

Pointer (Zeiger) speichern Speicheradressen von Variablen.

Grundlegende Pointer-Operationen

```
1 int var = 42;
2 int *ptr = &var;    // Pointer auf var
3
4 printf("%d", *ptr); // Dereferenzierung: Wert 42
5 *ptr = 100;         // Aendert var ueber Pointer
```

Zusammenhang zwischen Arrays und Pointern

Arrays und Pointer sind eng verwandt:

Array als Pointer

```
1 int arr[3] = {10, 20, 30};
2 int *ptr = arr;    // Aequivalent zu &arr[0]
3
4 printf("%d", *(ptr+1)); // Zugriff auf arr[1]
```

Pointer-Arithmetik

```
1 int *ptr1 = &arr[1];
2 ptr1++;    // Bewegt zum naechsten Element
```

Mehrdimensionale Arrays

```
1 int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
2 printf("%d", matrix[1][2]); // Ausgabe: 6
```

Pointer auf Pointer

```
1 int var = 5;
2 int *ptr = &var;
3 int **ptr_to_ptr = &ptr;
4
5 printf("%d", **ptr_to_ptr); // Ausgabe: 5
```

Dynamische Speicherverwaltung

```
1 int *dyn_arr = malloc(5 * sizeof(int)); // Allokierung
2 dyn_arr[0] = 10;
3 free(dyn_arr); // Freigabe
```

Strings als char-Arrays

```
1 char name[] = "Max"; // Automatisches Nullbyte '\0'
2 char *str = "Mustermann";
3
4 printf("%s %s", name, str); // Ausgabe: Max Mustermann
```

Vergleiche und Best Practices

Wichtige Unterschiede

Arrays	Pointer
Fester Speicherbereich	Kann auf beliebige Adressen zeigen
Größe bekannt	Kennt nur die aktuelle Adresse
Adresse nicht änderbar	Zeigerwert änderbar

Best Practices

- Immer Array-Grenzen beachten
- Pointer vor Dereferenzierung auf NULL prüfen
- Dynamisch allokierten Speicher immer freigeben
- Pointer-Arithmetik nur bei zusammenhängenden Bereichen
- const-Pointer für schreibgeschützten Zugriff

Eine schöne Kurzübersicht zu C findet sich unter <https://www.evamariakiss.de/tutorial/c-programming>